

Memory Management in OS

In a multiprogramming system, many processes stay in the **ready queue** (RAM) so the CPU never sits idle. This keeps the computer fast and responsive for users.

But to get this high performance, we must keep **several processes inside the main memory at the same time**. Since all these processes share the same RAM, the operating system must **properly manage the memory** so every process gets the space it needs and they don't disturb each other.

Logical Address vs Physical Address

1. Logical Address (Virtual Address)

This is the address **generated by the CPU** when a program runs.

Think of it as:

👉 **The address your program *thinks* it is using.**

- A logical address is used by the process (your program).
- It does **not** exist physically in RAM.
- You can access logical addresses (indirectly).
- The set of all these addresses is called **Logical Address Space** (0 to max).

Real-World Example:

Imagine you stay in a **hotel**.

You see **Room 101** on your key card — that is your *logical address*.

But inside the hotel system, Room 101 might actually be somewhere else in the building due to internal mapping — the *physical address*.

You never see that internal address.

You always use **101**.

2. Physical Address

This is the **real address in RAM** where your program's data actually lives.

Think of it as:

👉 **The actual location inside the computer's memory hardware.**

- The user/program **cannot directly access** physical addresses.
- It is present in the RAM chips physically.
- It is calculated by MMU.
- Physical Address Space is the set of all these actual memory locations.
- Range = $(R + 0)$ to $(R + \text{max})$, where **R = base address** in RAM.

Real-World Example:

Continuing the hotel example:

- Your room key says **Room 101** → Logical Address.
- But the hotel may internally map it to a corridor location:
 “Floor 2, Block B, Shelf 5” → Physical Address.

You don’t see this.
Only the **hotel management system** (like OS + MMU) works with this physical mapping.

3. MMU (Memory Management Unit)

MMU is a **hardware device** that converts logical addresses → physical addresses **at runtime**.

Think of MMU as:
👉 **The receptionist of the hotel**

You say: “Room 101”
Receptionist (MMU) knows the **actual physical room location** in the building and guides you there.

4. What the User Thinks vs What Actually Happens

- Your program believes it is running at logical address (like Room 101).
- But the OS + MMU place it at a physical memory location (like “Floor 2, Block B, Shelf 5”).
- This mapping keeps processes isolated, safe, and organized.

Super Simple

Concept	Meaning	Example
Logical Address	Address generated by CPU, not real in RAM	Room number on hotel key (101)
Physical Address	Actual location in memory	Real room location inside hotel
MMU	Converts logical → physical	Reception desk mapping key → room

How OS Keeps Processes Isolated and Safe (Memory Mapping & Protection)

Think of each process as a **tenant living in a separate flat** in a big building.
Each tenant must stay in their own flat and must not enter someone else's flat.

The OS makes sure every process stays in its own “memory flat.”

1. OS gives every process a Virtual Address Space (VAS)

This is like giving every tenant *their own map* of their flat.
They feel like they live alone, even though the building (RAM) is shared.

2. OS must know the legal memory area of each process

To keep processes separate, OS must know:

- **Where your process is allowed to live (start address)**
- **How much space it gets (size limit)**

This ensures a process cannot enter another process's memory.

3. Relocation Register (Base Address R)

This register stores the **starting physical memory address** of the process.

Example:

Process A is stored starting at physical address → **100040**

This is like saying:

“You live in Flat starting at Room number 100040.”

4. Limit Register (Logical Address Range)

This register stores **how big the process's memory is**.

Example:

Limit = **74600** means:

The process can access logical addresses from **0 to 74600**, nothing outside.

This is like saying:

“You can use only 74,600 sq. ft. area in your flat map.”

5. How MMU Protects Memory (Important!)

Whenever the CPU generates a logical address:

1. MMU first checks:

Is logical address < limit?

- If **yes** → **safe**

- If **no** → **illegal access**
2. If address is safe, MMU calculates the actual physical address:
physical = logical + relocation (base)

This ensures the process stays in its own memory flat.

6. During Context Switch (Process Change)

When CPU switches from one process to another:

- Dispatcher loads **new relocation register**
- Dispatcher loads **new limit register**

So each process always gets its own protected memory.

This is like changing the tenant's flat number every time a new person gets the keys.

7. What if a process tries to enter OS memory or another process's memory?

OS immediately raises a **trap (interrupt)**.

Meaning:

“Illegal memory access detected!”

OS treats this as a **serious error** and usually terminates the process.

Just like in a building:

If a person tries to open someone else's flat, security stops them instantly.

Real-World Analogy

OS Concept	Real-World Meaning
Virtual Address Space	Tenant's flat map
Relocation Register	Actual starting flat number
Limit Register	Size of tenant's flat
MMU	Security guard checking entry rules
Illegal access trap	Security alarm, tenant kicked out

Allocation Method on Physical Memory

When the OS loads processes into RAM, it can allocate memory in two ways:

a. Contiguous Allocation

The entire process sits in **one continuous block** of memory.

Like giving a family **one big room** in a hotel.

b. Non-contiguous Allocation

The process is split into **multiple pieces**, stored in different parts of memory.

Like giving a family **multiple smaller rooms** on different floors.

Contiguous Memory Allocation

In this method, **each process is stored in one single continuous block** in main memory.

a. Fixed Partitioning

The main memory is divided **beforehand** into fixed partitions.

Each partition can be the same size or different sizes.

Real-World Example:

Imagine a hotel where rooms are permanently built:

- Room A = 100 sq ft
- Room B = 200 sq ft
- Room C = 300 sq ft

Guests must fit inside these fixed rooms only.

Limitations of Fixed Partitioning

1. Internal Fragmentation

If a process is smaller than the partition size, the remaining space is wasted.

Example:

Partition = 200 MB

Process = 150 MB

Unused 50 MB → **internal waste**

Like a guest booking a 3-bed room but only using 2 beds.

2. External Fragmentation

Memory has free space, but it is broken into small scattered pieces, so you can't fit a big process even though total free space is enough.

Example:

Free spaces: 20 MB + 40 MB + 30 MB = 90 MB

Process needs 90 MB but **cannot fit** because memory is not continuous.

Like you have 3 small empty rooms in a hotel, but a big family needs one large room.

3. Limitation on Process Size

If the process is bigger than the largest partition, it **cannot** be loaded.

Example:

Largest partition = 300 MB

Process = 350 MB

→ Cannot load.

Like a family bigger than the largest room size — hotel cannot accommodate them.

4. Low Degree of Multiprogramming

Because partitions are fixed, only a limited number of processes can stay in memory, even if some partitions are empty.

Example:

Hotel has 10 rooms.

Even if 5 rooms are empty, you cannot break or resize rooms to fit more guests.

b. Dynamic Partitioning

- In this method, **memory partitions are created at runtime**.
- Means: **jab process load hota hai tabhi uske hisaab se memory ka block banaya jata hai**.
- Before that, memory is not divided into fixed blocks.

How it works

- A process arrives → OS checks how much memory it needs → OS creates a block of exactly that size.
- Next process arrives → Again OS creates a new block of required size.

So, **partition size = process size**.

Real-World Example

Example: Renting rooms in a guesthouse

Imagine you have a guesthouse with **no fixed-sized rooms**.

Whenever a guest arrives, you **build a room of exactly the size they need**.

- Guest A needs a small room → you build a small one.
- Guest B needs a big room → you build a big one.
- Guest C needs a medium room → you build that too.

This is **dynamic partitioning** — rooms (memory blocks) are created **on demand**.

Advantages over Fixed Partitioning

1. No Internal Fragmentation

Because every block is **exactly the size required**.

No empty space wasted inside a block.

Like: guest wants 1 bed, you give exactly 1-bed room → *no space waste*.

2. No Limit on Process Size

Process can be of **any size**, as long as total free memory is enough.

In fixed partitioning, if the largest room is small, big guest can't stay.

In dynamic → you build any size room.

3. Better Multi-programming

Memory is used **efficiently**, so more processes can fit.

No unnecessary empty spaces → more guests can stay.

Limitation – External Fragmentation

1. What is external fragmentation?

Over time, free memory gets scattered into **small free pieces**.

Example:

Rooms built and destroyed at different times →
empty spaces remain **between** occupied rooms →
these empty gaps can't be used for big guests.

Even if total empty space is enough,
because it's **not continuous**, process can't load.

Example of External Fragmentation

Guesthouse:

| Room A | Empty | Room B | Empty | Room C | Empty |

Total empty space = big.

But it is **broken into small pieces** →
a big guest can't fit.

This is **external fragmentation**.

Internal Fragmentation (Inside waste)

Wastage happens *inside* the allocated block.

The OS gives a block **bigger** than what the process actually needs.

Simple

Process needs 18 KB.

Partition size is fixed at 20 KB.

Process will use 18 KB → **2 KB inside is wasted**.

This leftover space **cannot be used by any other process**.

Real-World Example

Imagine you book a hotel room:

- You need space for **2 people**.
- The hotel only has **4-person rooms**.
- You and your friend stay there, but **2 beds remain empty**.

These empty beds are **internal wastage** →

because the room is yours, but you don't need the whole room.

This is **internal fragmentation**.

External Fragmentation (Outside waste)

Wastage happens *outside* the allocated blocks.

Memory is free, but it's in **small scattered chunks** so big processes cannot fit.

Simple

Total free memory = 100 KB
But it's in pieces like:

20 KB | 10 KB | 15 KB | 30 KB | 25 KB

If a process needs **60 KB**,
even though total free is **100 KB**,
it can't fit → because **no single continuous block** of 60 KB is available.

This is **external fragmentation**.

Real-World Example

Imagine a busy parking lot:

Car | Empty | Car | Empty | Car | Empty

Total empty space = **large**,
but the empty slots are **separated**.

A big SUV arrives →
even though total empty space is enough,
it doesn't fit in any one continuous slot.

These scattered empty parking spaces = **external fragmentation**.

Table

Type	Where is the waste?	Why it happens?	Real-world example
Internal Fragmentation	Inside the allocated block	Block > process size	Hotel room bigger than your need (empty beds inside your room)
External Fragmentation	Outside the blocks, in free spaces	Free memory is scattered	Parking lot with scattered empty spots

